Workflow Agentic Automation

Complete Documentation

Upstream Prevention • Detailed Diagnostics • End-to-End Flow

📚 Contents

1. **14.** Executability Principle — EVERYTHING must be executable
2. **15.** Test Failure Reporting — Structured diagnosis
3. **16.** Assertion Design — Prevent red flags
4. **17.** Traceability Record — AUTOMATION-CHECKLIST per TC
5. **18.** End-to-End Flow — PRD → TSD → TC → Automation Live

14 — Executability principle: No assumptions

Goal: EVERYTHING must be EXECUTABLE in dev and stg. No aspirational behavior. Every layer verified in real environment before claiming automation.

Core Rule

```
NOT: "Assume the code will work, test later"

BUT: "Verify everything works in REAL environment, then write code"
```

What Must Be Executable

```
 Layer  Must Be Executable  Verify By
```

```
|-------|-------------------|-----------|

 **SPEC**  test.step, expect() runs in real env  Run test in dev/st
 **USECASE**  Endpoint call works, response valid  Grounding: endpo
 **Layer 3**  Selector found, element exists  Grounding: live inspe
 **FACTORY**  Generated data insertable  Test: data passes validati
 **PRE**  Setup endpoint works, DB mutates  Grounding: endpoint exi
 **Environment**  Dev and stg have SAME structure  Verify: endpoint
```

Per-Layer Executability Rules

Layer 1: SPEC

✅ **Executable:**

```
test('@covers DF-M1-POS-001', async () => {

  const data = factory.generateEmployee()
```

```
  const result = await usecase.execute(data)

  expect(result.status).toBe('DRAFT')  // Actual value, not assum

})
```

❌ **NOT executable (assumption):**

```
test('Create employee', async () => {

  // Assume usecase.execute works

  const result = await usecase.execute({ name: 'John' })

  expect(result).toBeDefined()  // Assume something will be d

})
```

Checklist:

- [ ] Usecase call = real function (not mock)
- [ ] Expected value = from TC (not from code)
- [ ] Assertion = meaningful, can fail
- [ ] No assumption of success

Layer 2: USECASE

✅ **Executable:**

```
export class CreateEmployeeUsecase {

  async execute(data: EmployeeInput): Promise {

    // Call REAL endpoint
```

```
    const res = await http.post('/v1/employees', data)

    // Validate response (not assume success)
    if (!res.ok()) {
      throw new Error(`API error: ${res.status()}`)
    }
    return { success: true, data: res.json() }
  }
}
```

❌ **NOT executable (assumption):**

```
export class CreateEmployeeUsecase {

  async execute(data) {

    // Assume endpoint exists (not verified)

    // Assume response has data field

    const res = await http.post('/v1/employees', data

    return res  // Assume caller will handle error

  }

}
```

Checklist:

- [ ] HTTP call = to real endpoint (not mock)
- [ ] Response validated = check status, structure

- [ ] Error handling = explicit throw (not silent)
- [ ] No assumption of success

Layer 3: PAGE/SCREEN/COMPONENT

✅ **Executable:**

```
// src: kerjaan-client-portal/src/pages/CreateEmp

async fillName(name: string) {

  // Selector verified in grounding (live inspect

  await this.page.fill('[data-testid="employee-na

  // Wait for actual network (not fixed sleep)
```

```
  await this.page.waitForLoadState('networkidle')
}
```

❌ **NOT executable (assumption):**

```
async fillName(name: string) {
  // Assume selector exists (not verified)
  await this.page.fill('[data-testid="employe
  // Assume 2 seconds is enough (hardcoded wa
  await this.page.waitForTimeout(2000)
}
```

Checklist:

- [ ] Selector = grounded (live inspected app
- [ ] Selector = commented with // src:
- [ ] Wait = dynamic (waitForLoadState, not w
- [ ] No assumption of element existence

FACTORY

✅ **Executable:**

```
export function generateEmployee(override

  // Generate varied, valid data
```

```
  return {
    name: faker.person.fullName(),
    email: faker.internet.email(),
    salary: faker.number.int({ min: 30000
    ...overrides
  }
}
```

❌ **NOT executable (assumption):**

```
export function generateEmployee() {
  // Hardcoded same data every run
```

```
  return {
    name: 'John Doe',
    email: 'john@test.co',
    salary: 5000000  // Assume this i
  }
}
```

Checklist:

- [ ] Data = varied (not hardcoded)
- [ ] Data = schema-valid (email form
- [ ] No API call in FACTORY
- [ ] No assumption of data uniquenes

PRE (Precondition)

✅ **Executable:**

```
export async function preconditio

  // PRE endpoint verified in gro

  const response = await request.

    role: 'manager',

    email: `mgr-${Date.now()}@tes

  })

  // Assert PRE actually worked
```

```
  expect(response.status()).toBe(

  expect(response.json().id).toBe

  return response.json()

}
```

❌ **NOT executable (assumption)

```
export async function precond

  // Assume endpoint exists (

  // Assume it always succeed

  const response = await requ

  return response  // No asse
```

```
}
```

Checklist:

- [ ] PRE endpoint = verified
- [ ] PRE response = validate
- [ ] PRE data = unique (not
- [ ] PRE assertion = explici

Environment Parity

Rule: All layers tested

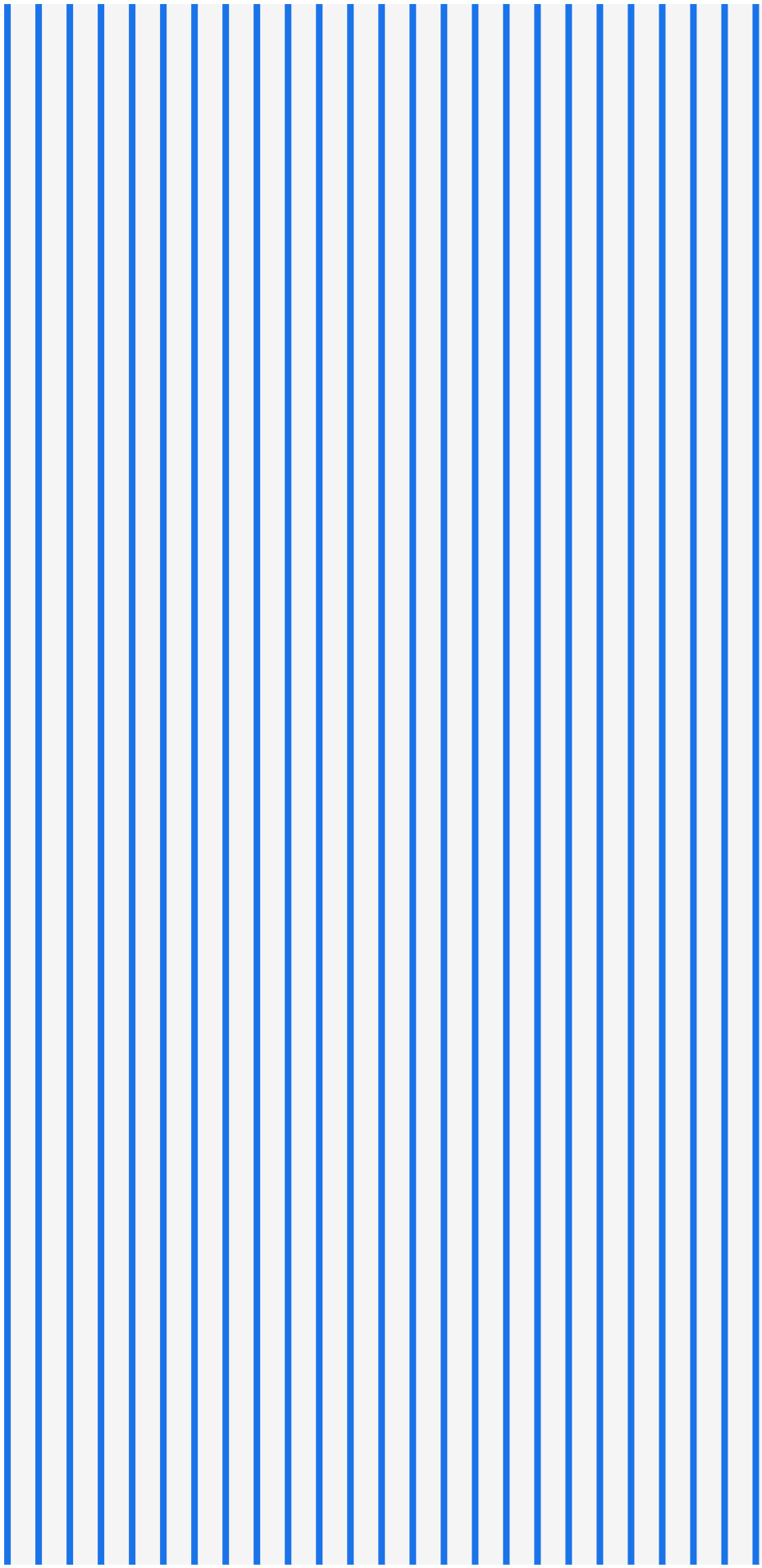
❌ WRONG:

- PRE tested in dev ✓
- Scenario tested in stg ✗
- Endpoint different: dev l
- Test fail, unclear why

✅ RIGHT:

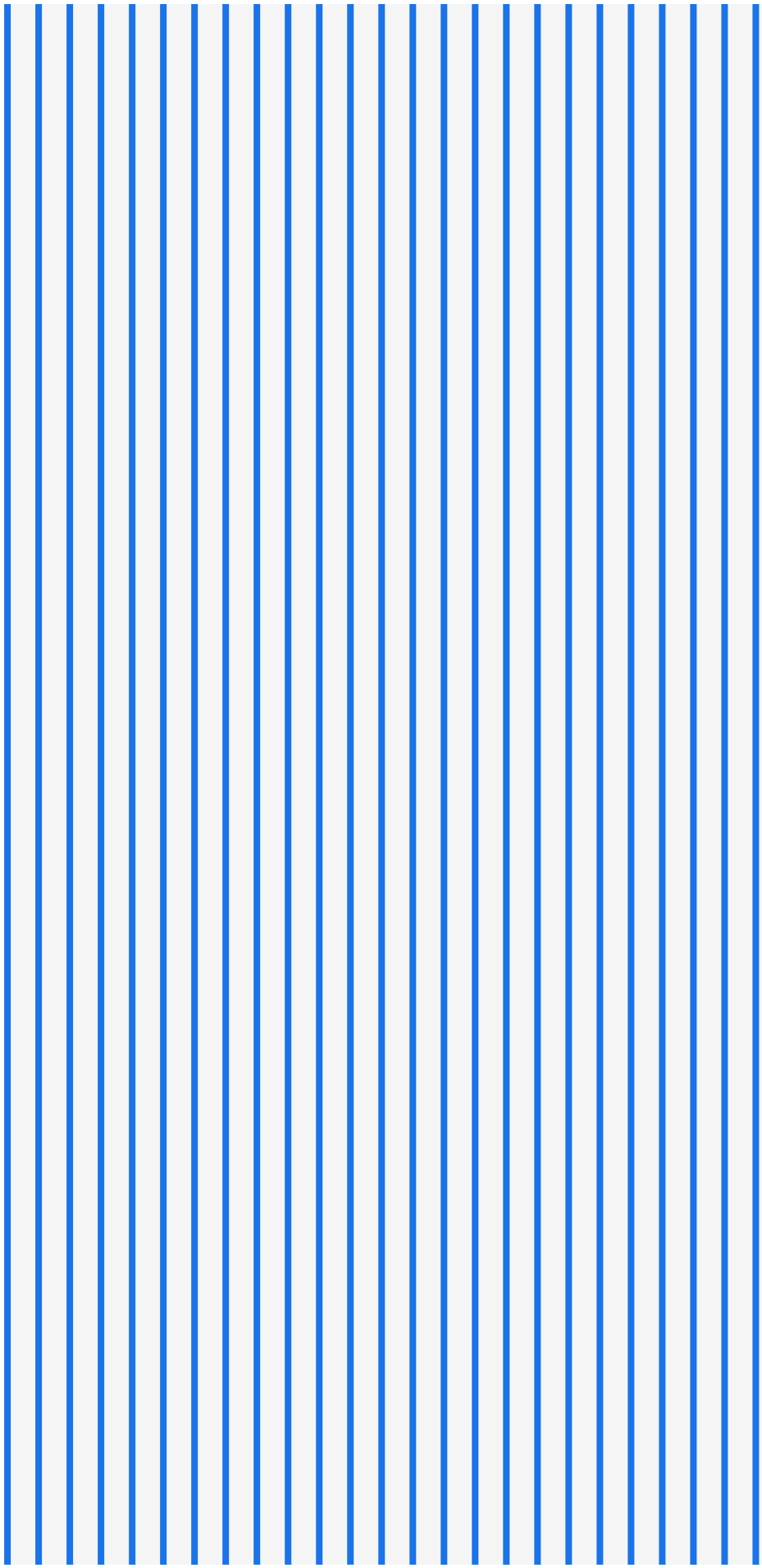
- PRE tested in dev
- Scenario tested in dev
- OR both in stg
- Same endpoint, same schem

Verify:



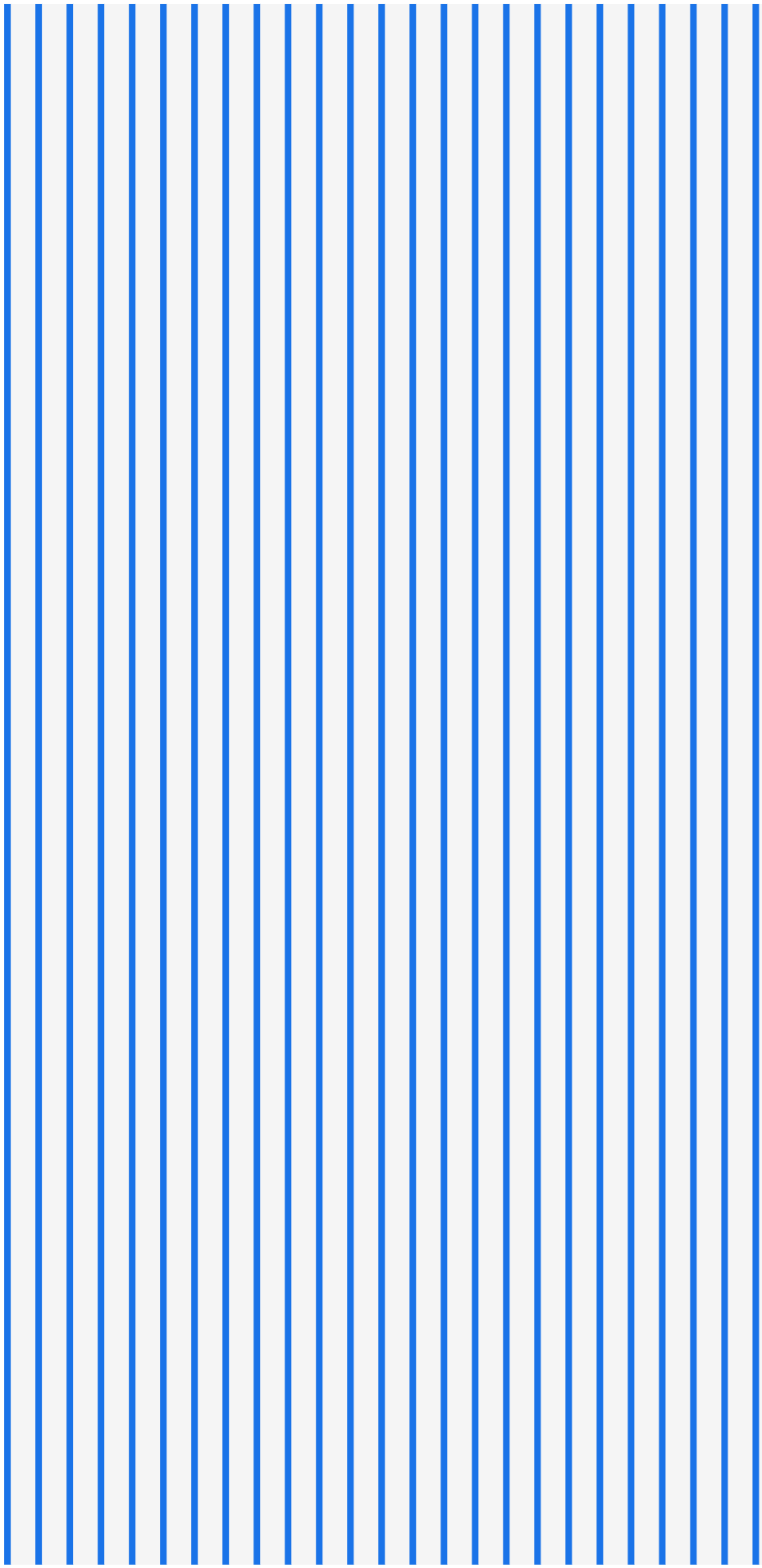
Check endpo

```
echo $API_ENDPOINT  #
```



Check envir

```
grep -r "localhost\|s
```

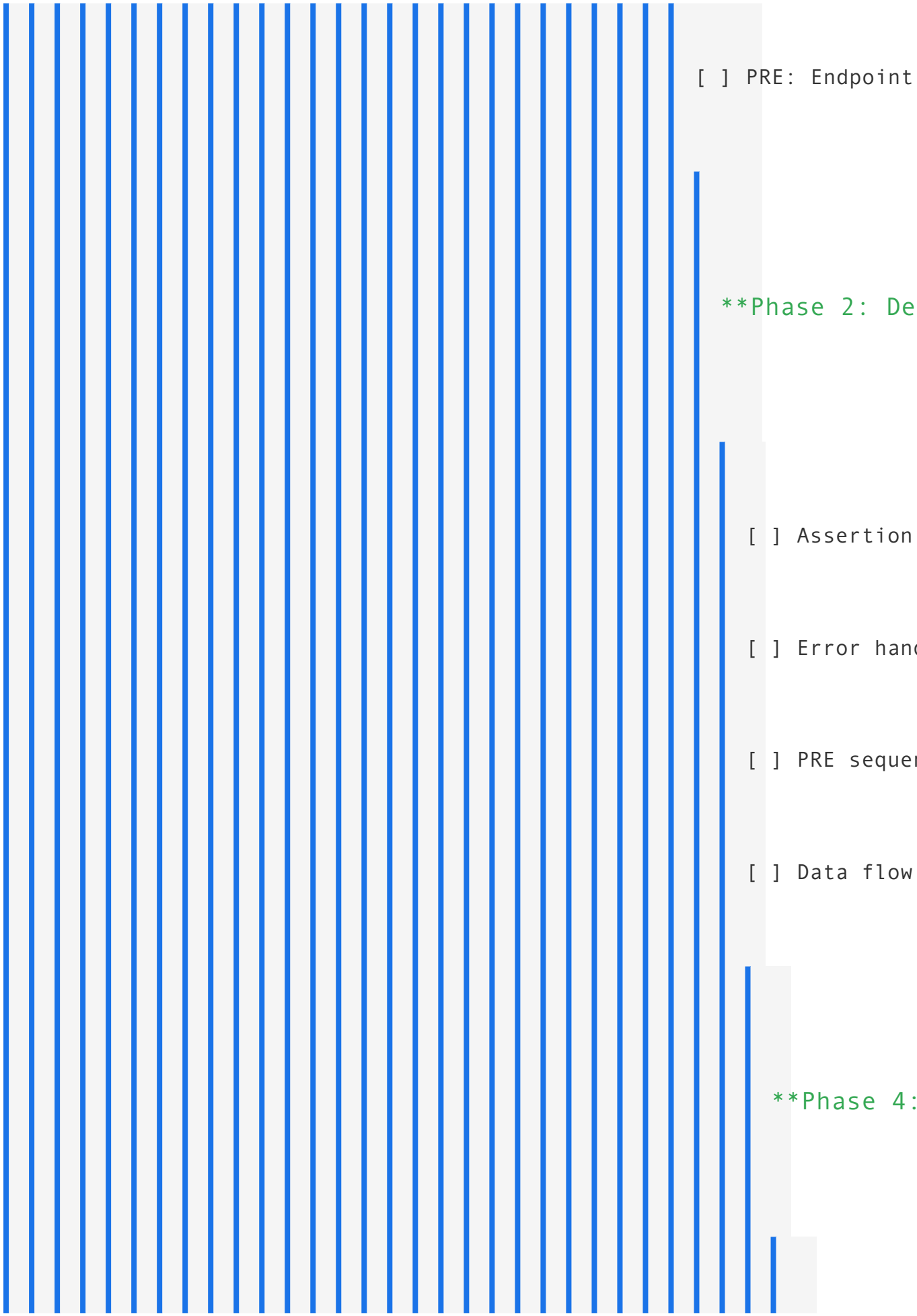


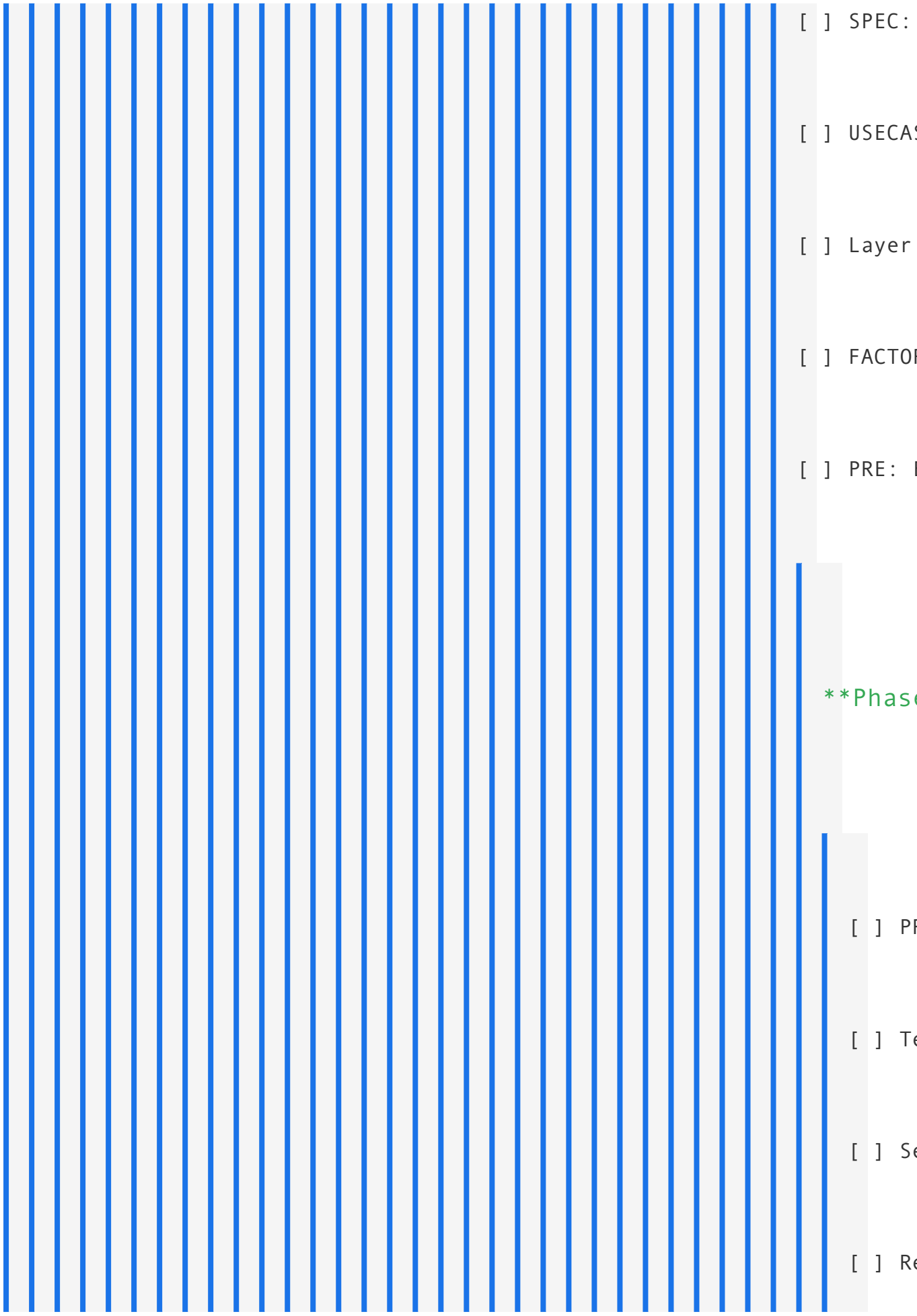
Test in tar

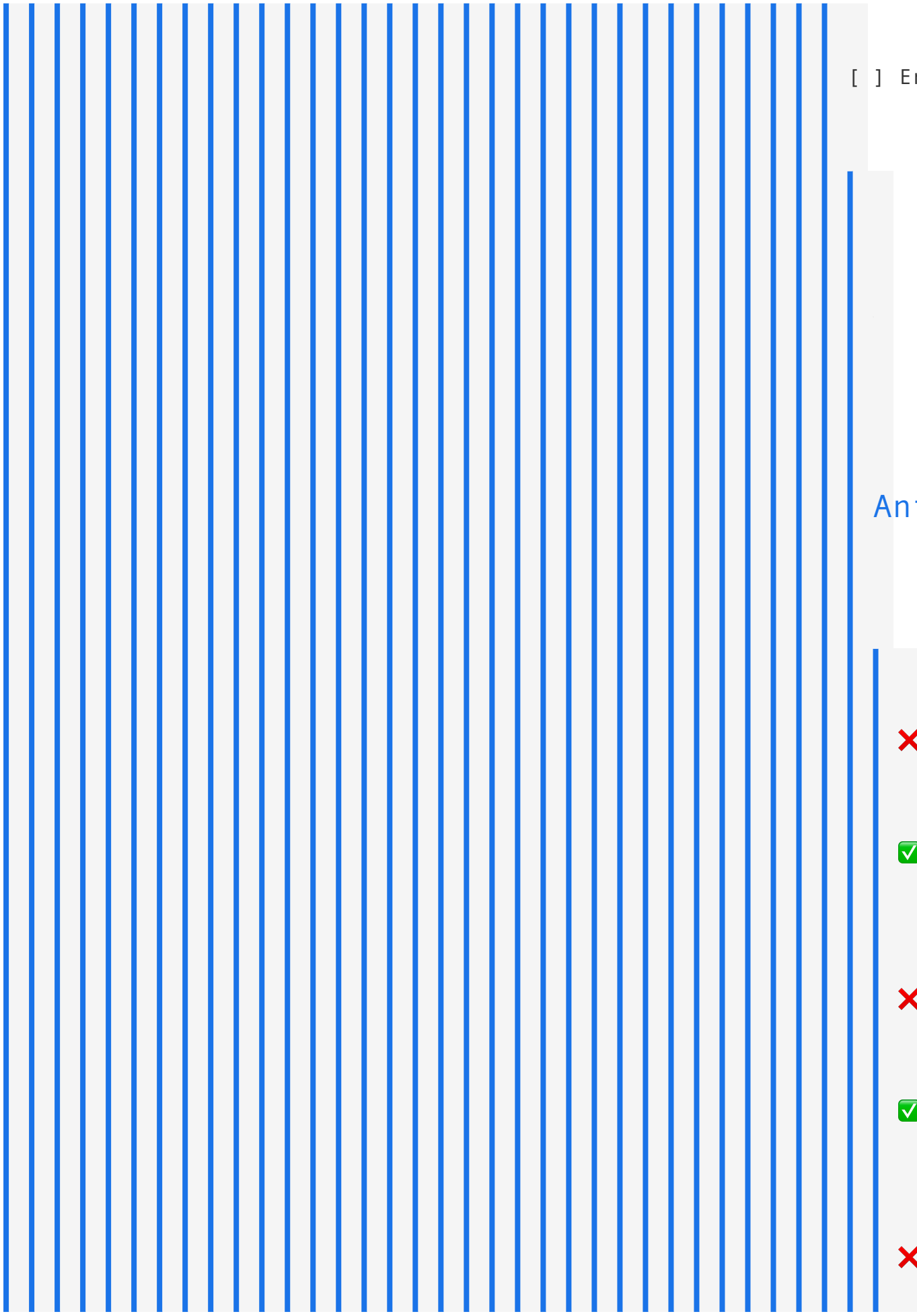
```
ENVIRONMENT=dev npm t

ENVIRONMENT=stg npm t
```


Not mixed!
Checklist: Ex
**Phase 1: Ground
[ ] Selector: Liv
[ ] Endpoint: Sea
[ ] Expected valu
[ ] Error scenari


[ ] PRE: Endpoint
**Phase 2: De
[ ] Assertion
[ ] Error hand
[ ] PRE sequen
[ ] Data flow
**Phase 4:


[ ] SPEC:
[ ] USECA
[ ] Layer
[ ] FACTO
[ ] PRE:
**Phas
[ ] PR
[ ] Te
[ ] Se
[ ] Re


[ ] E
An
❌
✅
❌
✅
❌

